Memoria FIFO

Proiect PSN

CRISTEA LEONARDO
GRUPA 30217
ÎNDRUMĂTOR:ING.DIANA POP

Conținut

1 Specificatii …….2
2 Proiectare……3
2.1 Schema Bloc……3
2.2 Unitatea de Control și Unitatea de Execuție……3
1.2.1 Maparea intrărilor și ieșirilor cutiei mari pe cele 2 componente UC și UE 4
1.2.2 Determinarea resurselor(UE).....4
2.2.3 Schema bloc a primei descompuneri……6
2.2.4 Schema de detaliu a proiectului…….9
3 Manual de utilizare și întreținere…….9
4 Justificarea soluției alese……..9
5 Posibilități de dezvoltări ulterioare……..9
6 Bibliografie……..9

MEMORIA FIFO

1 Specificații

Implementarea **memoriei FIFO** folosind documentația din cartea „**PROIECTAREA SISTEMELOR NUMERICE FOLOSIND TEHNOLOGIA FPGA**”, autori S.Nedevshi, Z.Baruch și O.Creț.Vom prezenta în continuare un model de implementare a unei memorii de tip FIFO(coadă),viteza sa de operare este dublă în cazul implementării cu memorii RAM sincrone față de cele asincrone.Modul de lucru cu port dual ne permite scrierea și citirea simultană a datelor,ceea ce multiplică de 4 ori viteza memoriei FIFO față de implementările cu memorii asincrone ,de 2 ori datorită modului de scriere sincron și încă de două ori datorită modului de realizare dual-port.Circuitul folosit în continuare este o memorie FIFO 16(locatii de memorie) x 8(număr de biți a unei valori) care poate fi utilizată pentru o interfață PCI(o interfață PCI cuprinde 2 memorii FIFO,una de scriere și una de citire,care sunt utilizate ca tampoane pentru transferuri burst mode,instantanee una dupa alta succesiva).Utilizatorul va introduce niște valori,care vor reprezenta intrările ,iar după ce utilizatorul nu a mai introdus nicio valoare sau memoria s -a umplut(nu mai avem loc,e plina) se vor aprinde cateva led-uri care vor reprezenta în funcție de poziția lor valorile care au fost introduse în ordinea cronologică. Memoria trebuie să funcționeze după cum urmează:

-Starea inițială este una de așteptare,memoria așteaptă să primească intrări

-Folosind butonul “**DATA_IN**” și “**PUSH**” vom introduce valorile care dorim in coada ,după cum îi spune numele **FIFO**(first in first out) valorile vor fi afișate în ordinea în care au fost citite,dacă la un moment dat ne-am oprit din a introduce valori ,iar după un timp am reluat procesul,valorile vor fi un introduse începând cu ultima **poziție** care a fost ocupată(**pointer**) ,daca memoria este plina aceasta **comanda** nu va avea niciun efect deoarece nu va șterge primul element pentru a face loc următorului element ce va fi introdus sau altfel de stergeri,pur și simplu nu va introduce

-Elementele vor fi introduse în memorie folosind un **pointer** pentru adresa (**Numarator**),acest principiu aplica si pentru stergerea lor din memorie

-Butonul “**POP**” este folosit pentru afișarea și ștergerea totodată din memorie a valorii din poziția (**pointerul**) de la începutul cozii,daca memoria este vida(nu avem niciun element in ea) aceasta comanda nu va fi executată din moment ce nu exista vreo valoare

-Pentru afișarea elementelor șterse se va folosi “**DATA_OUT**” ,care este practic un set de 8 led-uri menite sa indice valoarea ce a fost ștearsă

3

2 Proiectare

2.1 Schema Bloc

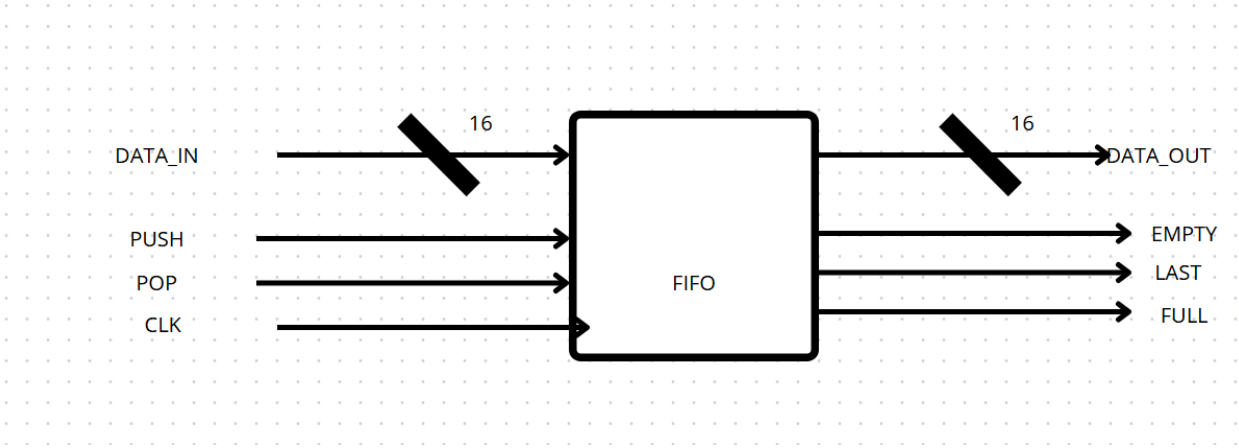


Figura 1 Cutia neagră a sistemului cu intrarile si iesirile stabilite

Acest prim pas poate fi unul complicat,exista intrari si iesiri care pot sa nu fie specificate în text dar pe care proiectantul trebuie sa le adauge pentru a putea implementa pe placa(ex.: buton "**ok**" ,pentru confirmarea valorilor introduse.Acest buton nu este specificat în text dar el trebuie să fie).

Proiectantul trebuie la acest pas să identifice use case-urile proiectului(adică ce acțiuni se pot face),să ia fiecare caz si să l parcurgă cu imaginația pas cu pas,preferabil pe placa,ținând cont de următoarele restricții (placa ***BASYS 3*** *16 comutatoare,5 butoane,16 led-rui,4 SSD*).Astfel dacă exista acțiuni ascunse ,implicite, proiectantul trebuie sa le descopere și sa le treaca fie la intrari fie la ieșiri dacă e cazul.

Oricand se mai pot adăuga semnale de intrare si iesire pe parcurs.

2.2 Unitatea de Control și Unitatea de Execuție

2.2.1 Maparea intrarilor si iesirilor cutiei mari pe cele 2 componente UC si UE.

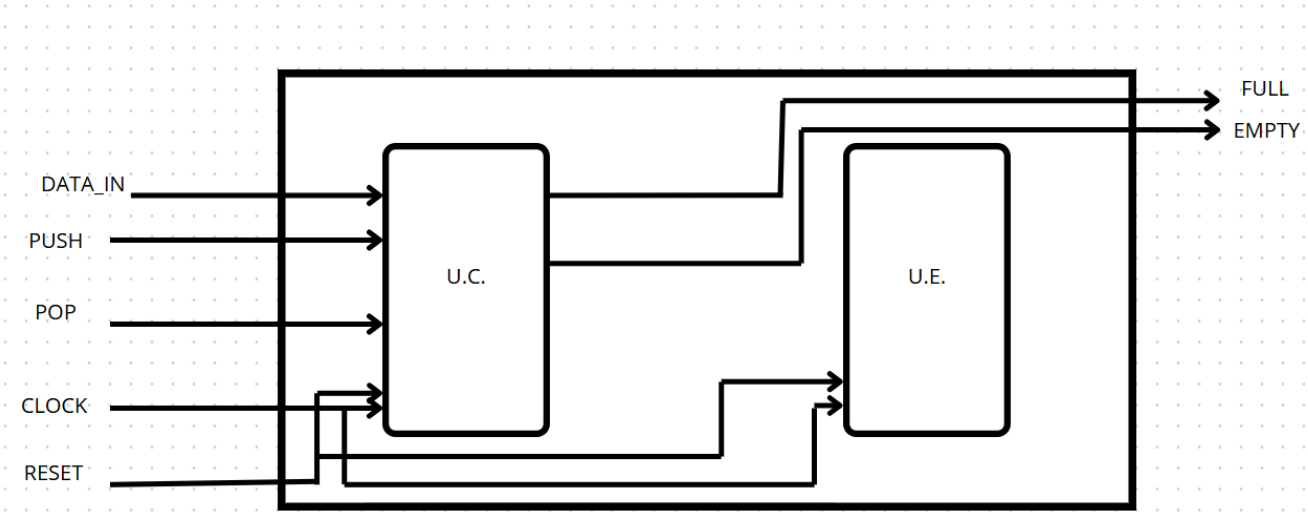


Figura 2 Maparea intrarilor si iesirilor cutiei negre pe intrarile si iesirile componentelor din prima descompunere

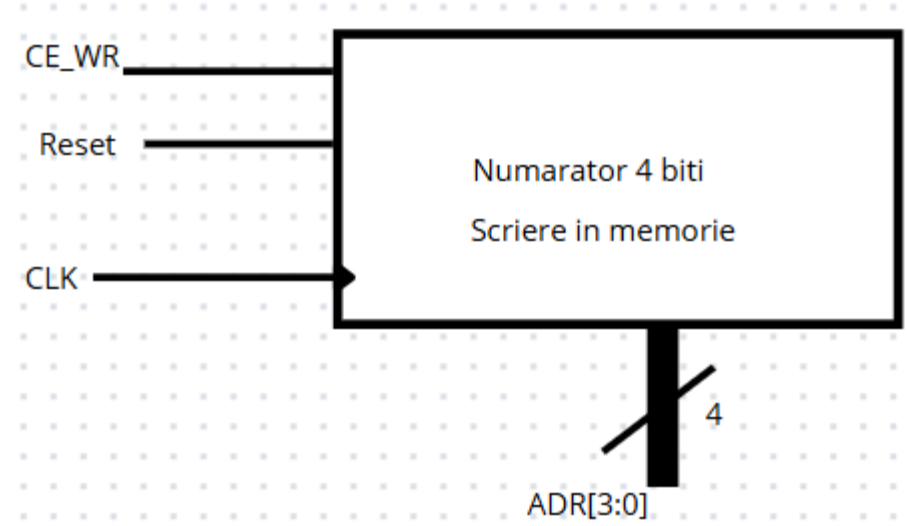
Putem împărți atat intrările cat și ieșirile în 2 categorii :de date și de control.Aceasta separare este esențială la început.

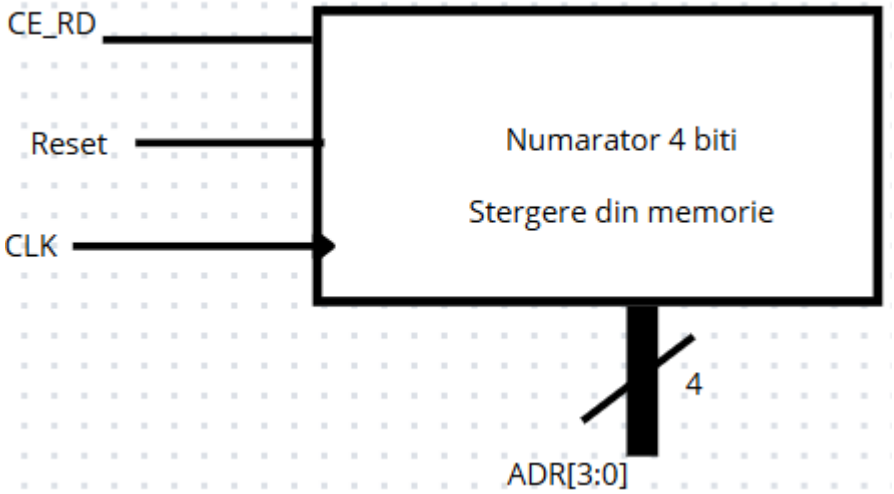
2.2.2 Determinarea resurselor(UE)

RESURSE:

1.2 x Numaratoare de cate 4 biti pentru pointeri adresa(RADR1,RADR2)

Acesta este un numarator pe 4 biti.Acest circuit va avea o iesire ce va indica adresa unde se va executa ,fie stergerea ,fie adaugarea in memorie.In plus unul opereaza pentru ștergerea celălalt pt adaugare,la fiecare semnal de ceas ,cand trebuie sa se efectueze operația corespunzătoare se va aduna ,rezultand calcularea corecta a adresei unde va fi se va efectua operatia. 5

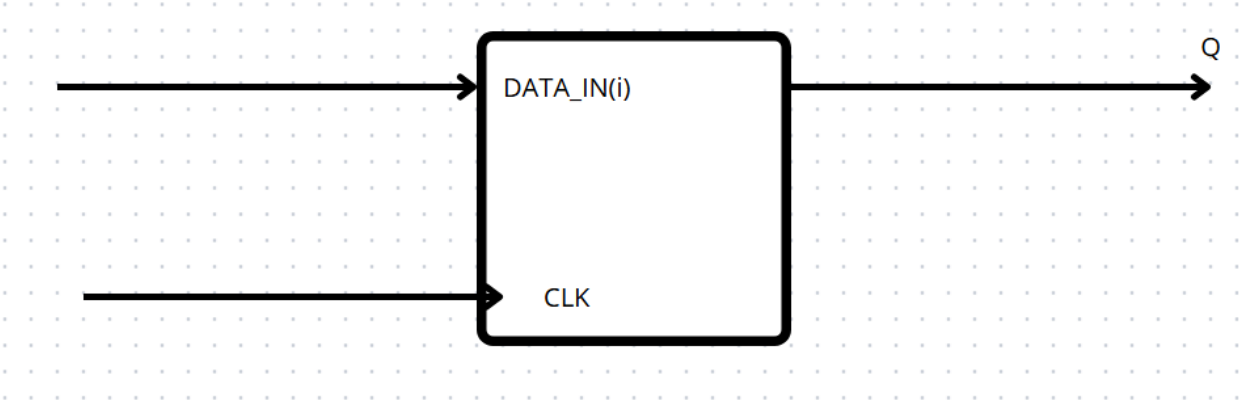




2.Bistabilul D

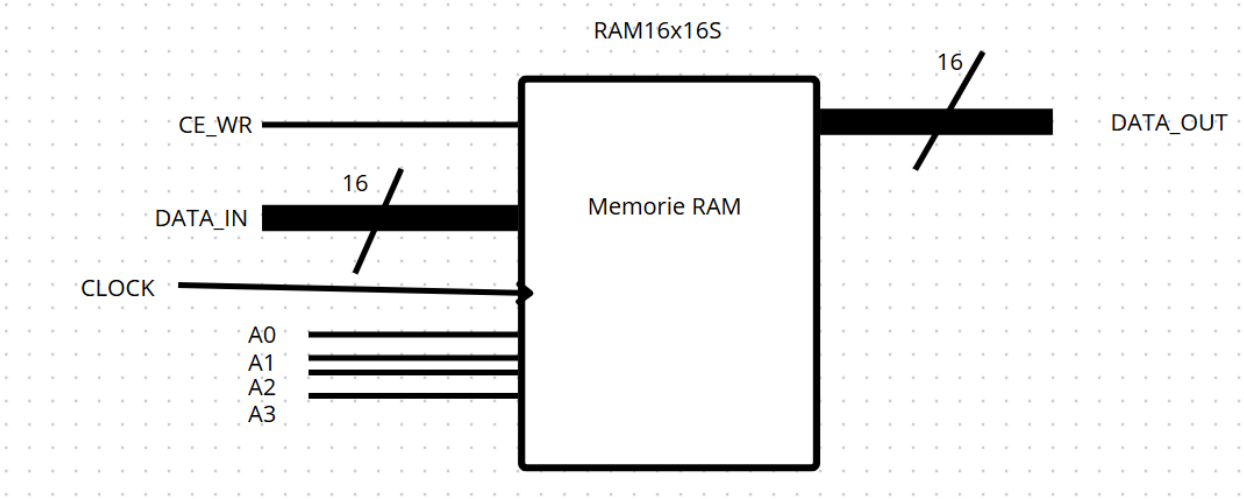
Folosim 16 bistabile pentru a introduce valoarea in Memoria RAM.

6



3Memoria RAM 16 locatii * cate 16 de biti RAM16x16S

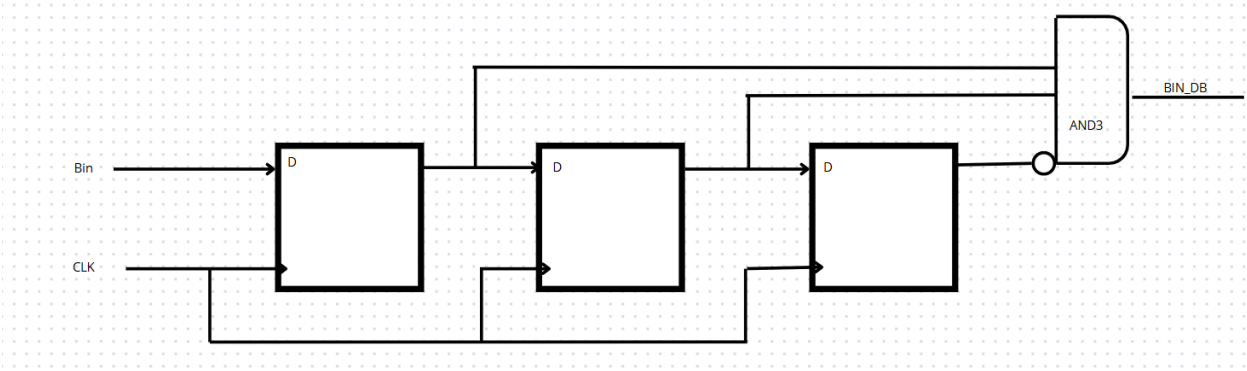
Aceasta memorie o folosim pentru a stoca și a șterge valorile din coada în funcție de operația dorită,valorile sunt salvate într-o adresa specifica ordinii în care acestea au fost citite(proprietate a cozii).



7

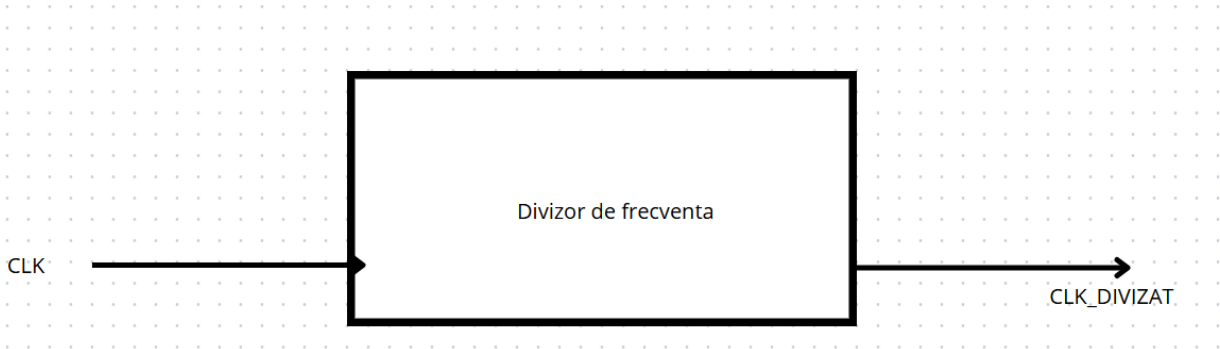
4 DEBOUNCER/MPG

Folosim debouncerul pentru ca la o apasare datele sa fie introduse o singura data nu de mai multe ori, vrem sa generam un mic impuls pe o perioada mica de ceas~ 10 ms timp suficient pentru a putea sa introduca valoarea in RAM.



5Divizor de frecventa

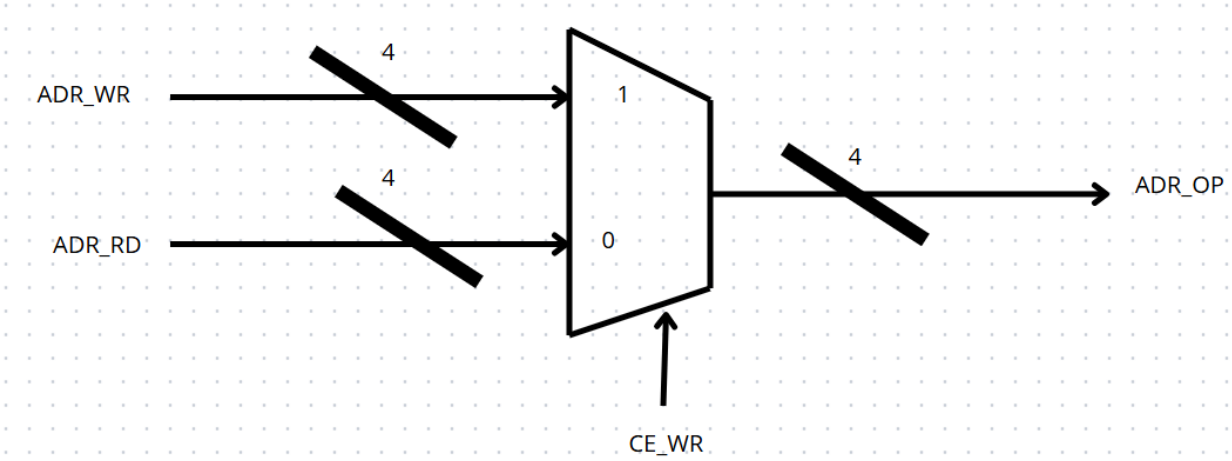
Aceasta componenta o vom folosi pentru afisarea digitilor folosind o frecventa ~100Khz pe anozi si pe catozii de la seven segment display.



8

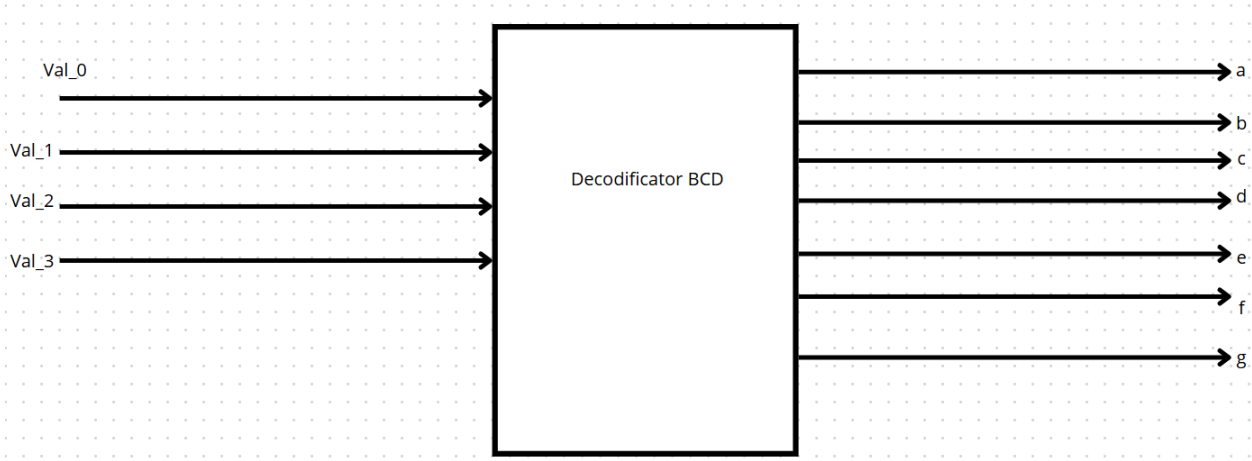
6Multiplexor 2_1 pe 4 biți

Acest multiplexor il folosim pentru a selecta din memoria adresa de la operatia care dorim sa facem sa stergem(0) si sa afisam sau sa scriem(1)



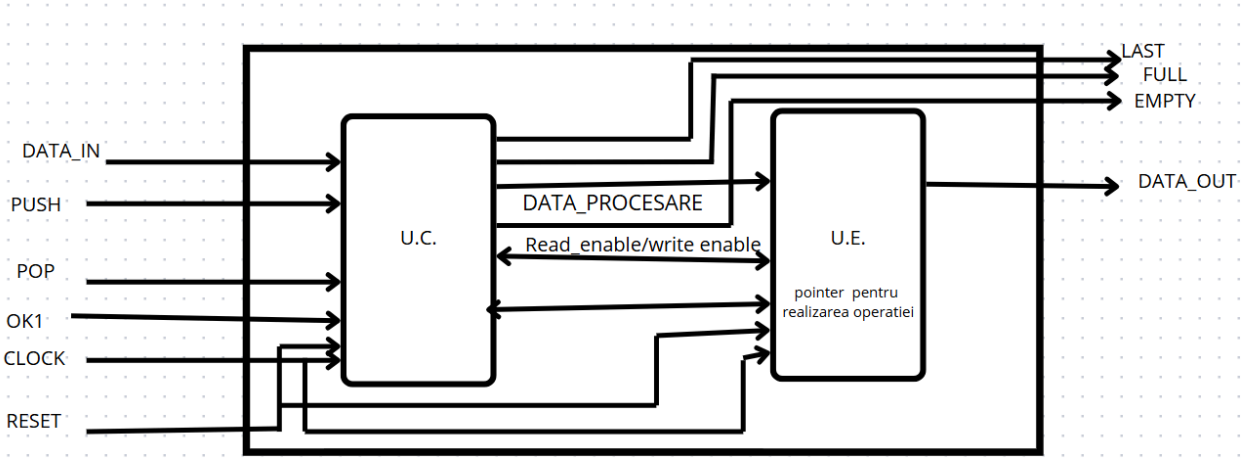
7 Decodificator

Folosim decodificatorul pentru a transforma valorile din binar in hexa pentru a putea fi afisate pe Seven segment display



9

2.2.3 Schema bloc a primei descompuneri



2.2.4 Schema de detaliu a proiectului

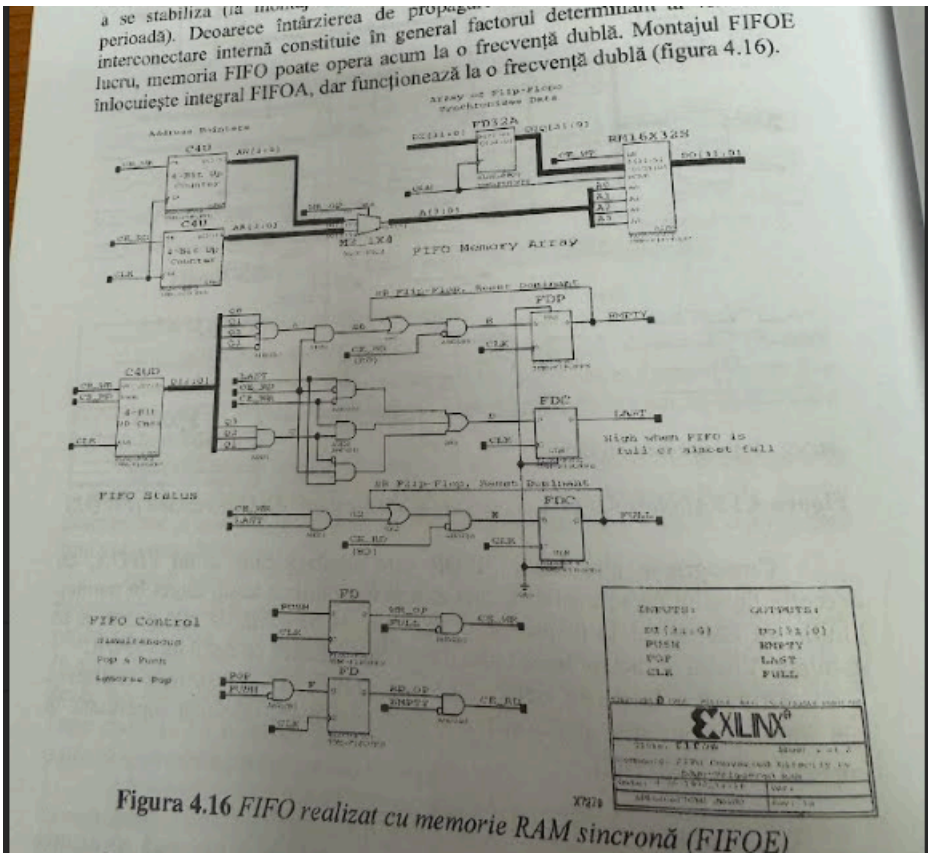
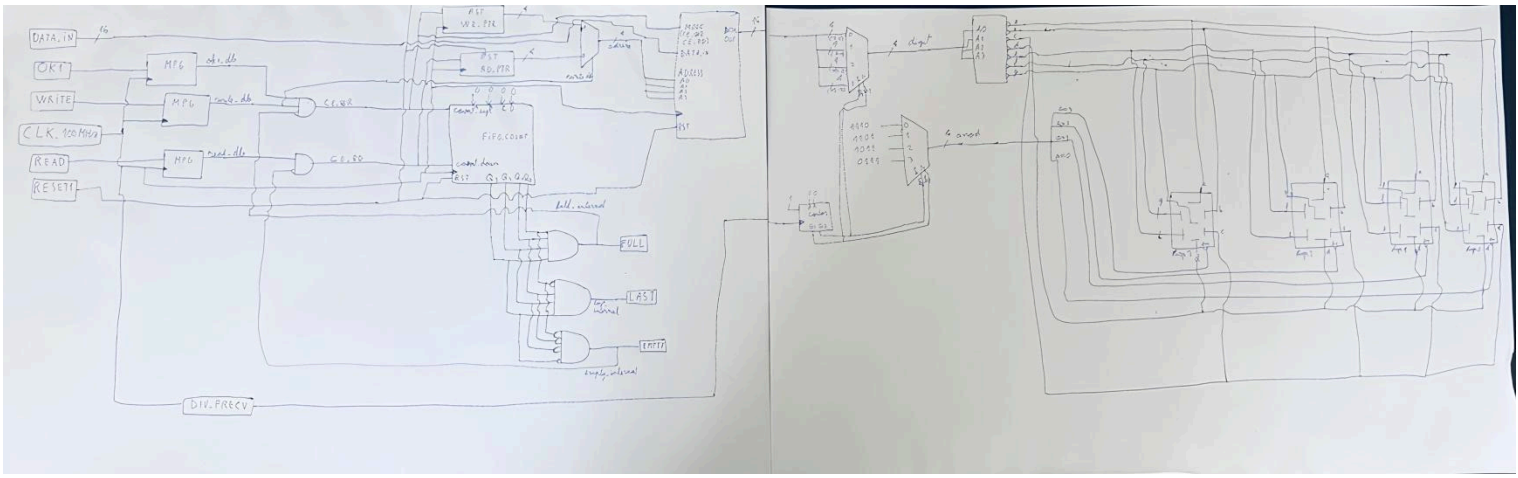


Figura 4.16 *FIFO realizat cu memorie RAM sincronă (FIFOE)*

10

Schema preluata din cartea lui O.Cret.



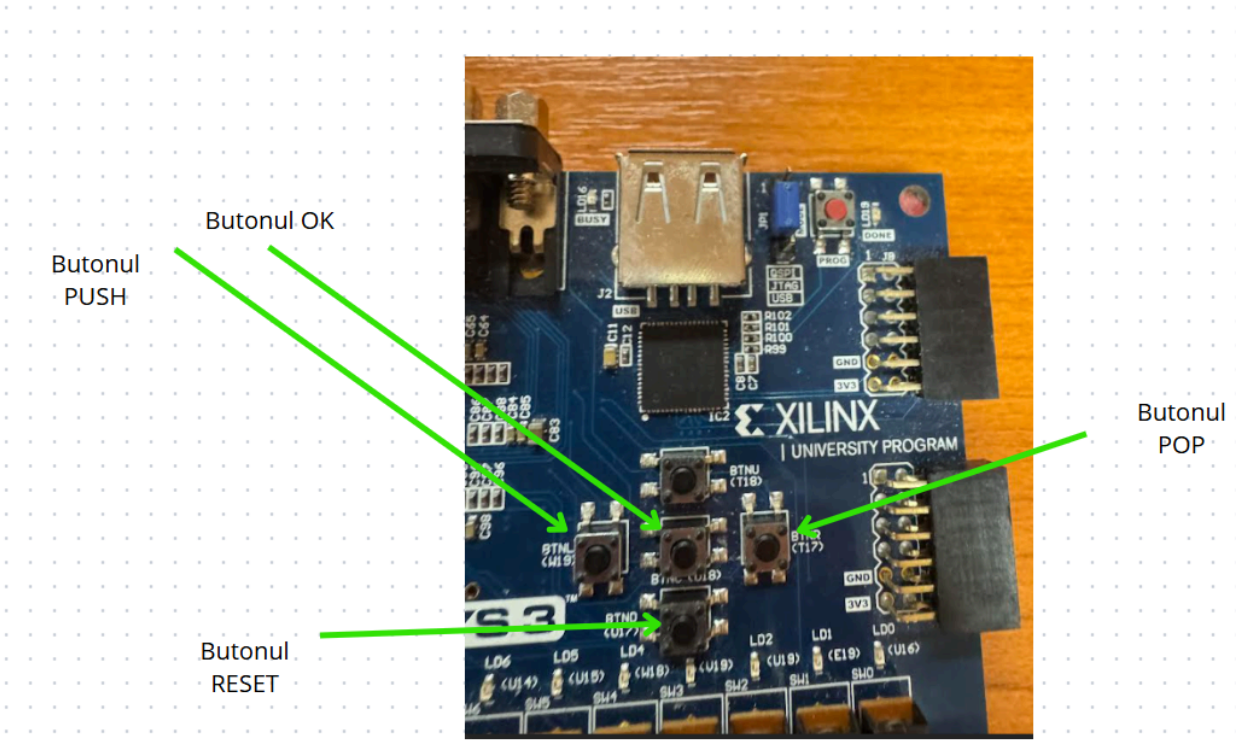
Schema circuitului realizat pe 2 foi A3

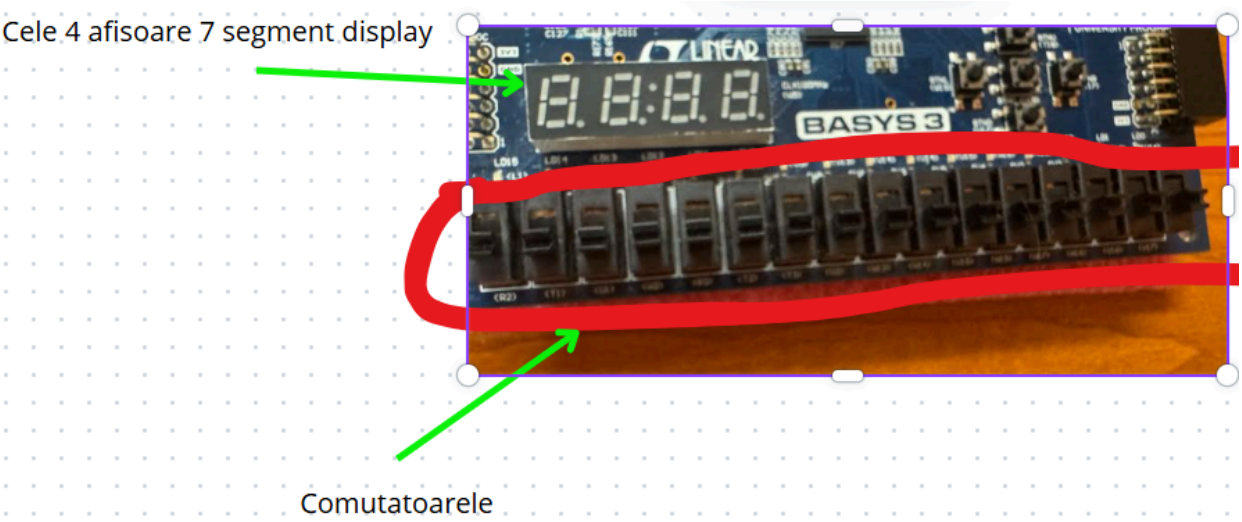
Proiectul meu-[proiect]

11

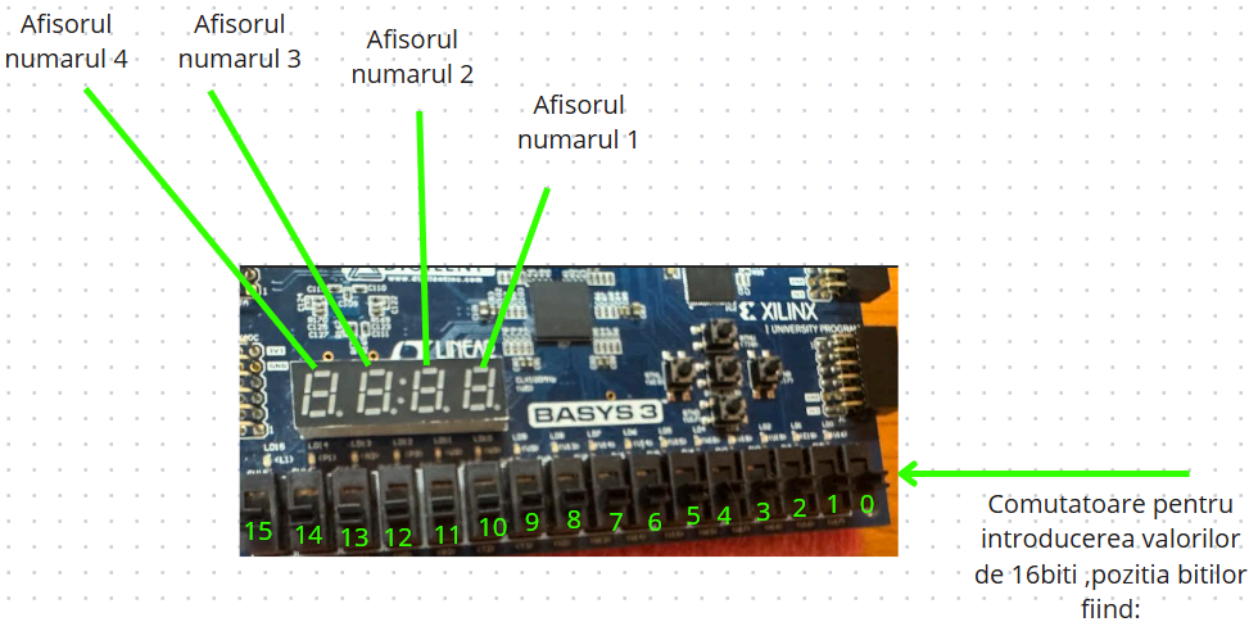
3 MANUAL DE UTILIZARE

Butoane pentru folosirea memoriei fifo:





12



Primul afisor va afisa valoarea introdusa in hexazecimal(baza 16) a comutatoarelor:0-3

Al doilea afisor va afisa valoarea introdusa in hexazecimal a comutatoarelor:4-7

Al treilea afisor va afisa valoarea introdusa in hexazecimal a comutatoarelor:8-11

Al patrulea afisor va afisa valoarea introdusa in hexazecimal a comutatoarelor:12-15

Instructiuni de utilizare

-folosind comutatoarele impreuna cu butonul de **push**(introducerea a valorii) si butonul **OK**(butonul de confirmare a introducerii valorii) ne arata ce valoare am introdus in **baza 16** (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F) in **seven segment display** .Apasam mai intai butonul de **push** si mentinem apasat pana ce apasam si butonul de **OK** dupa care lasam de apasat cele 2 butoane.Ledul cel mai din dreapta va fi iluminat semn, ca memoria e complet goala,dupa introducerea primei valori becul se va stinge astfel ca am introdus o valoare in memorie.

-Introducem valori folosind **comutatoarele** pana cand se ilumineaza al doilea buton din dreapta semn ca mai putem introduce o singura valoare totodata odata apasate butoanele de **push** si **ok** in aceasta ordine ne va afisa pe **seven segment display** valoarea pe care am introdus o

- Introducem inca o valoare si se va aprinde al treilea led din dreapta semn ca memoria e **plina** ,nu ne va lasa sa mai adaugam nimic in memorie. 13

- Daca apasam **pop** ne va afisa in ordinea in care am introdus in memorie valorile ,si se va aprinde cel de al doilea led din dreapta si daca vom continua sa apasam pana cand becul din dreapta se va aprinde vom afisa fiecare valoare pe care am adaugat o,cand am afisat ultima valoare un vom mai putea sa stergem nimic din memorie(evident) .

-Apasarea butonului **RESET** va sterge complet memoria,prin urmare va urma aprinderea ledului din dreapta care indica ca memoria e goala

Constrangeri:Nu se vor putea introduce mai mult de 16 valori in memorie.

Memoria atunci cand e **goala** nu va afisa nimic decat 0000 sau ultima valoare care a fost citita

Ledul din dreapta(**EMPTY**) se aprinde cand avem 0 elemente in memorie,ledul al doilea(**LAST**) din dreapta se aprinde cand avem 15 elemente introduse in memorie ,iar al treilea led(**FULL**) din dreapta se aprinde cand am introdus cele 16 valori(memoria e **plina**).

14

4 Justificarea solutiei alese

Am ales aceasta configuratie deoarece este mai usor realizarea sa sincrona comparativ cu versiunea sa asincrona si pentru ca are performante relative duble fata de memoria asincrona(**viteza**).

Acest proiect m-a ajutat sa inteleg mai bine cum functioneaza atat memoria **RAM** cat si memoria **FIFO** introducand valori ce vor fi stocate in memorie ce apoi urmeaza a fi afisate.

Realizarea acestui proiect putea fi facut folosind un registru universal de 16 biti in loc sa folosim o memorie RAM cu toate acestea folosirea unei memorii FIFO cu registre implica o constructie mai usoara a programului ,folosind registrii se poate accesa cam orice locatie din memorie a registrului,nu neaparat consecutiv ,tocmai de aceea si pentru a face folos de memoria RAM am decis sa implementez memoria FIFO in acest mod.

15

5 Posibilitati dezvoltari ulterioare

-afisarea in baza 10 a numerelor atat cand sunt introduse cat si cand sunt citite

Acest lucru ar avea drept consecinta limitarea utilizatorului a numarului maxim pe care l poate introduce la 9999(10) (adica de maxim 14 biti si fara sa se treaca de 9999) in loc de FFFF(16) ,adica 65535(10).Cu toate ca acest lucru este realizabil ,implica folosirea unui **decodificator** care sa se ocupe de fiecare cifra in parte care nu este greu de facut

-memorarea a mai multor valori

Acest lucru este usor de solutionat trebuie doar marirea capacitatii memoriei ram,folosirea unui alt **multiplexor** care primeste adrese pe 5 biti sau mai multi(asta daca vrem cumva sa memoram doar valori de 16 biti considerand ca avem 16 comutatoare) iar conditia pentru **pointerii** de semnal de **LAST** si **FULL** se vor modifica astfel ca **LAST** o sa fie **activ** cand contorul de elemente de memorie o sa fie egal cu 2^5 -1(nr.de biti de adresa) si **FULL** cu 2^5

16

6 BIBLIOGRAFIE

Cartea ,,Proiectarea sistemelor numerice folosind tehnologia FPGA” scrisa de Zoltan Baruch, O.Cret, editura MEDIAMIRA,1999

Cursul ,,Proiectare Logica’’ sustinut de dr.ing. Dragos Florin Lisman

Cursul ,,Proiectarea sistemelor numerice’’ sustinut de prof.Vlad Miclea

Youtube: https://www.youtube.com/watch?v=Nr8q5VW-mXI&t=478s
https://www.youtube.com/watch?v=fqUuvwl4QJA

17